

설비 고장을 예측하는 AI 데이터 분석

고장 임박 분류 모델

Part 2 · 라벨 생성 및 데이터 준비

C O N T E N T S

목차

01 RUL 기반 고장 임박 문제

02 라벨 생성 및 데이터 준비

라벨과 클래스 · x·y 구성 · 데이터 분리

03 랜덤 포레스트 고장 임박 분류

02

라벨 생성 및 데이터 준비

RUL에서 정답을 만들고, x 와 y 를 가르고, 학습/평가로 나누는 흐름

모델에게 가르칠 '정답'이 필요한 이유

이번 모듈의 첫 일은 모델이 향할 **정답(라벨)**을 직접 만드는 일

01 · 지도 학습

입력과 정답이 짝지어진 데이터로 배우는 방식

02 · 정답 없음

무엇을 향해 학습할지 알 수 없음

03 · 이번 모듈의 목표

그 정답(라벨)을 직접 만드는 일

우리 데이터의 상황

RUL이라는 숫자는 있지만, 0/1 정답은 직접 만들어야 함

있음

데이터가 가진 숫자

RUL — 고장까지 남은 cycle 수

없음

아직 만들어지지 않은 정답

곧 고장인가 아닌가의 **0/1 정답**

해야 할 일

RUL을 근거로 0/1 정답을 직접 만들기

파생 라벨 — 기존 컬럼으로 새 정답 만들기

있는 컬럼에서 규칙으로 새 정답을 끌어내기

정의

파생 라벨 = 원래 있던 컬럼에서 규칙으로 만들어 낸 새 정답

"파생"은 무언가로부터 끌어낸다는 뜻 — 기존 컬럼에서 새 정답을 끌어내는 일

우리 문제에서

RUL 숫자를

곧 고장(1) / 괜찮음(0)으로 바꾸기

실무에서

데이터를 분석 목적에 맞게

가공하는 흔한 작업

파생 라벨의 다른 예

일상에서도 흔한 원리 — 숫자를 둘로 가르는 일

01 · 시험 점수

원본 값 → 합격/불합격

60점 이상이면 **합격(1)**
아니면 **불합격(0)**

경계가 되는 값 **60점**

02 · 기온

원본 값 → 더움/보통

28도 넘으면 **더움(1)**
아니면 **보통(0)**

경계가 되는 값 **28도**

03 · RUL

원본 값 → 곧 고장/괜찮음

30 이하면 **곧 고장(1)**
아니면 **괜찮음(0)**

경계가 되는 값 **30 cycle**

경계가 되는 값을 **임계값(threshold)**이라 부름

임계값(threshold) 개념

넘느냐 못 넘느냐를 가르는 문턱 같은 값

01 · 정의

0과 1을 가르는 경계가 되는 기준 숫자

02 · 우리 문제

RUL의 경계값

03 · 규칙

임계값보다 작으면 1(곧 고장), 크면 0(괜찮음)

임계값은 분석가가 정하는 선택

데이터가 주는 값이 아니라 **우리가 결정하는 값**

01 · 경계

한 값이 전체를 결정

임계값 하나로 전체 라벨이 결정됨

단일 숫자의 **큰 영향**

02 · 선택

데이터가 알려 주지 않음

데이터가 주는 게 아니라 우리가 정하는 값

분석가의 **판단**

03 · 근거

근거를 두고 결정

현장 사정과 목적에 맞게 신중히 결정

현장 사정과 **목적 반영**

RUL ≤ 30 → 1, failure_soon 규칙

임계값 30 — 이하에 등호 포함 (정확히 30이면 1)

규칙 01

RUL이 30 이하이면 → **failure_soon = 1**

규칙 02

RUL이 31 이상이면 → **failure_soon = 0**

경계 처리

30이라는 임계값을 기준으로 둘로 나눔 (30은 1쪽 포함)

규칙 적용 예시

RUL 30이 1로 분류되는 점을 확인 — 경계값 자신의 소속이 헛갈리기 쉬움

RUL	판정	failure_soon
150	넉넉함	
45	아직 여유	
30	경계(이하)	
8	임박	

150·45 → 0, 30·8 → 1 — 임계값 30 자체는 1쪽 포함

임계값을 바꾸면 생기는 변화

같은 데이터라도 임계값에 따라 라벨이 크게 달라짐

임계값 ↑ 50

1이 되는 행이 늘어남
더 일찍, 더 자주 경고

임계값 ↓ 20

1이 줄어들음
임박한 것만 경고

핵심 정리

어느 쪽이 맞을지는 **현장 사정**에 따라 결정

보수적 기준과 민감한 기준

고장 비용과 점검 비용을 견줘 균형점을 찾는 **분석가의 판단**

01 · 높은 임계값

예: 50

미리 많이 경고

놓침은 적지만 **헛점검** 늘 수도

안전하지만 점검 부담 ↑

02 · 낮은 임계값

예: 20

임박한 것만 경고

헛점검은 적지만 **놓칠 위험**

점검 부담 ↓ 그러나 위험 ↑

03 · 선택

분석가의 판단

고장 비용과 점검 비용을 견줘 결정

정답이 따로 없는 **균형점 찾기**

현장 맥락에 따라 결정

'임박' 기준은 현장마다 다르다

임계값은 현장 맥락이 결정 — 일률적인 기준이 없음

01 · 설비 종류

무엇을 다루는가

**큰 설비는 여유 있게
소모품은 임박해서**

교체 난이도가 다름

02 · 고장 비용

멈췄을 때 손실

**멈추면 손실 큰 설비는
일찍 경고**

라인 전체 정지 위험 고려

03 · 정비 리드타임

부품 조달 시간

**부품 조달이 길면
미리 경고**

조달 2주 → 적어도 2주 전 경고

개념을 잡았으니, 다음은 이 규칙을 **코드**로 어떻게 적는지 확인

조건으로 라벨 만들기

조건식의 참/거짓을 `astype(int)`로 1/0으로 변환

CODE · LABEL.PY

```
# RUL이 30 이하인지 묻는 조건식 → 참/거짓  
# astype(int)로 참은 1, 거짓은 0으로 변환  
  
df['failure_soon'] = (df['RUL'] <= 30).astype(int)
```

이 한 줄의 의미

각 행마다 $RUL \leq 30$ 판정 → `failure_soon` 새 컬럼에 저장

한 줄에 담긴 두 단계

참/거짓 판정 → 0/1 변환 → 새 컬럼 저장

STEP 1 · 조건식

참/거짓 만들기

```
(df['RUL'] <= 30)
```

각 행이 **True** 또는 **False**로 판정됨

조건 판정 단계

STEP 2 · 변환

숫자로 바꾸기

```
.astype(int)
```

참 → **1**, 거짓 → **0**

숫자 변환 단계

STEP 3 · 저장

새 컬럼에 담기

```
df['failure_soon']
```

결과를 **새 컬럼**에 저장

이 두 값이 곧 클래스

클래스(class)란 — 0과 1이라는 두 묶음

분류에서 나뉜 각 묶음 — 우리 문제는 [이진 분류](#)

01 · 정의

분류에서 나누어진 각 묶음(범주)

02 · 우리 문제의 클래스

0(괜찮음)과 1(곧 고장)

03 · 이진 분류

클래스가 둘인 경우

각 클래스가 담는 것

진짜 찾고 싶은 쪽은 **클래스 1 — 곧 고장**

01 · 클래스 0

괜찮음

RUL이 넉넉한 평상시 시점들

평상시 운전 시점의 묶음

02 · 클래스 1

곧 고장

RUL이 임계값 이하인 시점들

고장 임박 시점의 묶음

03 · 관심 대상

찾아내고 싶은 쪽

우리가 찾고 싶은 쪽은 클래스 1

"곧 고장날 시점"이 진짜 목표

value_counts로 클래스 개수 세기

각 값이 몇 번 나오는지 세는 함수 — 0과 1의 개수 확인

CODE · COUNTS.PY

```
# 각 클래스(0, 1)가 몇 개인지 카운트  
df['failure_soon'].value_counts()
```

CHECKPOINT

0의 개수와 1의 개수 중 어느 쪽이 훨씬 많은지 확인

결과 출력 직후 직관적으로 차이가 드러남

개수 결과 읽기

0이 훨씬 많은 결과 — 클래스 불균형 상태

01 · 일반적 결과

보통 0(괜찮음)이 1(곧 고장)보다 훨씬 많음

02 · 이유

고장 임박 시점은 전체 운전 기간 중 일부뿐

03 · 다음 주제

이렇게 한쪽이 크게 많은 상황 — 클래스 불균형

클래스 불균형이란 무엇인가

한 클래스가 다른 클래스보다 훨씬 많은 상태

01 · 정의

한 클래스가 다른 클래스보다 훨씬 많은 상태

02 · 구분

다수 클래스(0) vs 소수 클래스(1)

03 · 우리 데이터

0이 1보다 많은 전형적 불균형

다수 클래스와 소수 클래스

하필 찾고 싶은 쪽(1)이 **적은 쪽**이라는 곤란함

01 · 다수 클래스

많은 쪽

우리 문제의 0(괜찮음)

평상시 시점 — 데이터 풍부

02 · 소수 클래스

적은 쪽

우리 문제의 1(곧 고장)

드물어 데이터 부족

03 · 핵심

역설적 상황

찾고 싶은 건 적은 쪽인 소수 클래스

드물어 더 중요한데 데이터는 적음

불균형이 왜 문제가 되는가

"전부 0" 모델도 정답률 95% — 그러나 정작 중요한 1을 단 하나도 못 잡음

함정 01

무조건 다수(0)라고만 답해도 겉으로는 자주 맞음

함정 02

하지만 정작 중요한 곧 고장(1)을 놓치게 됨

대응

소수 클래스를 제대로 맞히도록 신경 쓰는 일

불균형이 부르는 위험

뒤에서 배울 **stratify**가 비율을 지키는 핵심 장치

01 · 착시

겉모습은 좋아 보임

전체 정답률은 높아 보여도 속은 비어 있음

숫자에 속지 말 것

02 · 놓침

중요한 것을 놓침

소수인 곧 고장을 못 잡음

예방의 본래 목적 실패

03 · 대비

분리 단계의 장치

데이터를 나눌 때 비율을 지키는 장치 필요

핵심 도구 **stratify**

라벨 분포를 그림으로 확인하기

숫자보다 그림이 불균형을 더 직관적으로 전달

01 · 시각화의 힘

숫자보다 그림이 불균형을 한눈에 보여 줌

02 · 도구

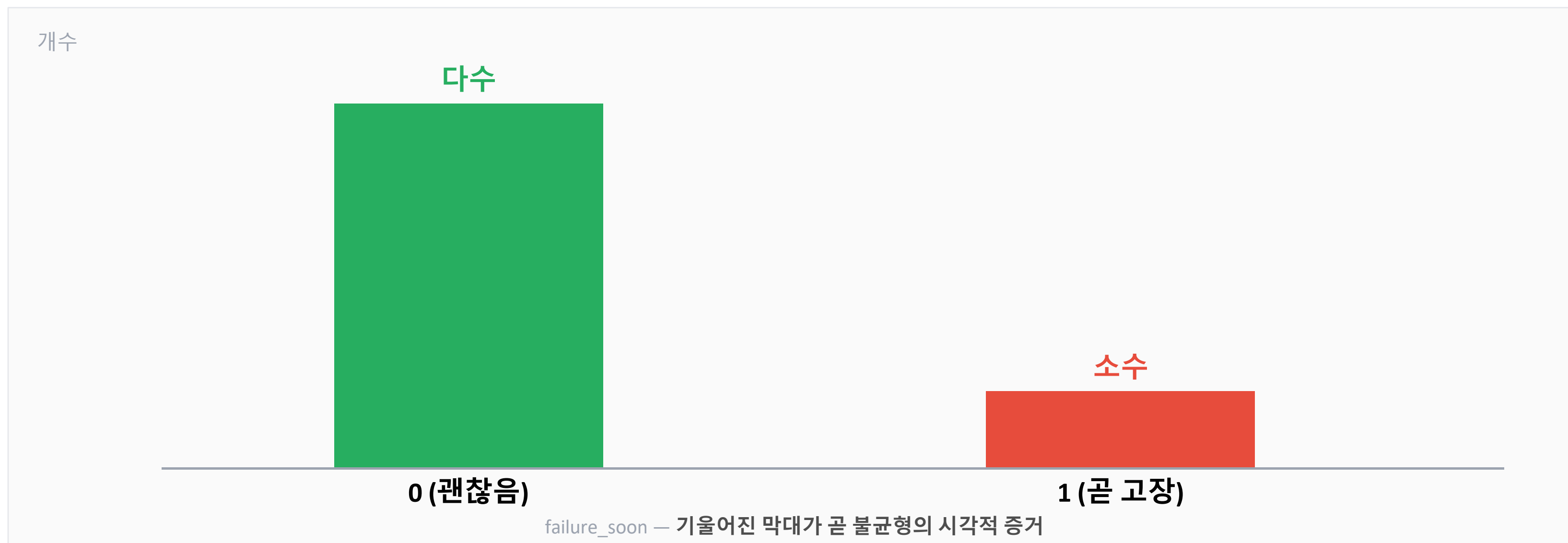
막대그래프로 0과 1의 개수를 비교

03 · 효과

기울어진 막대를 보면 불균형이 직관적으로 느껴짐

라벨 분포 막대그래프

0(괜찮음)의 막대가 1(곧 고장)보다 훨씬 높음



feature(특징, x)란 무엇인가

모델이 들여다보는 입력 단서들 — 보통 대문자 x 로 표기

01 · 정의

모델에게 보여 주는 입력값 — 보통 x 로 표기

02 · 우리 문제

센서값들이 feature

03 · 역할

판단의 근거가 되는 단서들

feature는 판단의 단서

의사의 검사값이 feature — 결과를 알려 주는 단서여야 좋은 feature

01 · 의사 비유

검사값으로 판단

체온·혈압 같은 검사값이 feature

측정값이 곧 단서

02 · 우리 데이터

설비 측정값

**sensor_2, sensor_3,
sensor_4 ...**

센서값을 입력으로 사용

03 · 핵심

좋은 feature의 조건

**결과를 알려 주는 단서여야 좋은
feature**

결과와 연관성이 있어야 함

target(정답, y)란 무엇인가

모델이 맞혀야 할 정답 — 보통 소문자 **y**로 표기

01 · 정의

모델이 맞혀야 할 정답 — 보통 **y**로 표기

02 · 우리 문제

failure_soon⁰ | target

03 · 학습 구조

feature(X)를 보고 **target(y)**을 예측

x와 y의 관계

학습 = x일 때 y가 무엇인지 데이터에서 관계를 찾는 일

01 · X (feature)

입력

센서값들

모델이 들여다보는 단서

02 · y (target)

정답

failure_soon

모델이 맞혀야 하는 답

03 · 학습

학습의 본질

x와 y의 관계를 모델이 익힘

데이터에서 패턴 찾기

x와 y를 나누는 이유

"이건 단서, 이건 정답"이라고 분명히 갈라 주어야 함 — [scikit-learn](#)도 x·y를 따로 받음

함께 두면

구분이 모호한 상태

모델이 입력과 정답을 구분하지 못함

어디까지 보고 어디부터 맞춰야 할지 알 수 없음

나눠 두면

역할이 분명한 상태

무엇을 보고 무엇을 맞춰야 하는지 명확

scikit-learn 도구도 x와 y를 따로 받도록 설계

x와 y를 나누는 작업

표에서 센서 컬럼만 x로, failure_soon만 y로 분리

x

입력

센서 컬럼들만 골라 담기

y

정답

failure_soon 컬럼만 담기

다음 슬라이드

어떤 컬럼을 x에 넣고 어떤 컬럼을 뺄지 프레임으로 정리

어떤 컬럼이 x이고 어떤 게 y인가

센서는 x · failure_soon은 y · 식별 컬럼과 RUL은 제외

컬럼	역할	모델 입력
sensor_2 ~ sensor_15	단서	x에 포함
failure_soon	정답	y
unit_id, cycle, RUL	식별 · 정답 단서	제외

RUL은 정답을 그대로 알려 주는 값이라 반드시 제외 — 그 이유는 곧 자세히 확인

x에서 빼는 컬럼들

각 컬럼이 x에 들어가지 않는 이유

01 · unit_id

이름표

설비 번호일 뿐 고장 단서가 아님

식별자 — 학습에 무의미

02 · cycle

의미가 일정치 않음

설비마다 의미가 달라 단서로 부적절

단위가 일관되지 않음

03 · RUL

정답의 출처

정답을 직접 알려 주는 값이라 빼야 함

failure_soon을 만든 그 값

x/y 구성 정리

이번 모듈에서 채택할 입력 x / 정답 y / 제외 컬럼

01 · 입력

x = 센서 6개 컬럼 (sensor_2, 3, 4, 7, 11, 15)

02 · 정답

y = failure_soon

03 · 제외

unit_id, cycle, RUL은 x에서 제외

정답이 새어 들면 안 된다

학습이 아니라 **커닝** — 시험 문제에 답을 적어 둔 셈

01 · 정의

정답 누수 — 정답 정보가 입력에 섞여 드는 일

02 · 우리 사례

RUL을 X에 넣으면 failure_soon을 거의 그대로 알려 줌

03 · 비유

시험 문제에 답을 적어 둔 것과 같음

누수가 부르는 착각

학습 만점 · 실전 무력 — 그래서 RUL은 반드시 x에서 제외

01 · 학습 단계

누수가 있으면 학습 때 성적이 비현실적으로 완벽

02 · 실제 운영

하지만 RUL을 모르는 실제 상황에서는 무력함

03 · 결론

그래서 RUL은 반드시 x에서 빼야 함

feature를 너무 많이/적게 넣으면

고장과 관련 깊은 센서들로 적당히 균형을 잡는 일

01 · 너무 많이

잡음 증가

관계없는 컬럼이 섞여 모델이 헛갈림

엉뚱한 패턴을 찾을 위험

02 · 너무 적게

근거 부족

단서가 부족해 제대로 판단 못 함

학습 근거의 결핍

03 · 적당히

중간 균형

고장과 관련 깊은 센서들로 균형

잘 골라 둔 6개 — 우리 수업 선택

적절한 feature 선택

잘 골라 둔 6개를 그대로 사용 — feature 선택 자체가 큰 주제

01 · 이번 수업의 선택

변화가 뚜렷한 센서 6개로 고정

02 · 깊은 주제

feature를 잘 고르는 일은 그 자체로 중요한 기술

03 · 입문 단계의 약속

지금은 준비된 6개를 믿고 사용

학습/평가 데이터를 나누는 이유

학습용으로 가르치고 평가용으로 시험

01 · 본질

모델의 진짜 실력은 본 적 없는 데이터로 재야 함

02 · 방법

데이터를 학습용과 평가용으로 나눔

03 · 사용법

학습용으로 가르치고 평가용으로 시험

시험 문제 비유

평가용 데이터는 학습 때 절대 보여 주지 않는 약속

01 · 학습용

train

공부에 쓰는 연습 문제

모델을 가르치는 자료

02 · 평가용

test

실력을 재는 시험 문제

미리 보여 주면 안 됨

03 · 보통 비율

실무 관행

학습 80, 평가 20 정도

우리 수업도 동일

평가 데이터는 학습에 쓰지 않는다

학습이 끝난 뒤 단 한 번, 실력을 썰 때만 사용

01 · 학습 중

평가용 데이터는 학습 과정에 단 한 번도 등장하지 않음

02 · 사용 시점

오직 다 배운 뒤 실력을 썰 때만 사용

03 · 의미

이 약속을 지켜야 평가가 공정함

평가의 공정함이 중요한 이유

공정한 평가가 곧 신뢰할 수 있는 모델 성능 추정치

01 · 잘못된 평가

평가용을 학습에 쓰면 성적이 부풀려짐

02 · 현장 결과

실제 현장 성능을 잘못 예상하게 됨

03 · 원칙

학습/평가 분리는 엄격히 지키는 약속

'외운 것'과 '이해한 것'의 차이

좋은 모델의 목표는 외우기가 아니라 **일반화**

01 · 일반화

이해한 학생

본 적 없는 데이터에도 잘 통하는 능력

새 문제에도 응용 가능

02 · 외우기

문제집을 통째로 외운 학생

**학습 데이터만 잘 맞히고 새 데이터엔
약함**

응용에 약함

03 · 목표

좋은 모델의 방향

외우기가 아니라 일반화

새 데이터에도 통하기

일반화를 확인하는 방법

학습 완벽 vs 평가 나쁨 — 외운 신호

01 · 나눠서 시험

측정의 기본

평가용 성적으로 일반화 정도를 가늠

학습/평가 점수 비교

02 · 경고 신호

위험 징후

학습은 완벽한데 평가가 나쁘면 외운 것

과적합 의심

03 · 대비

데이터 측면 대응

충분하고 다양한 데이터가 일반화를
도움

양과 다양성 둘 다

train_test_split() 사용법

x와 y를 넣으면 네 덩어리로 나눠 돌려주는 도구

CODE · SPLIT.PY

```
# scikit-learn에서 데이터를 나눠 주는 도구 불러오기
from sklearn.model_selection import train_test_split

# x와 y를 학습용/평가용으로 4덩어리로 분리
X_train, X_test, y_train, y_test = train_test_split(X, y)
```

반환 순서 약속

X_train, X_test, y_train, y_test 순서 고정

반환되는 네 덩어리

이름 끝 train = 학습용, test = 평가용

X_train

학습용 데이터

학습용 입력

가르치는 단서

X_test

평가용 데이터

평가용 입력

시험 문제

y_train

학습용 정답

학습용 정답

배우는 답안

y_test

평가용 정답

평가용 정답

채점 기준

test_size — 평가용 비율 정하기

보통 0.2 ~ 0.3 사이 — 우리 수업은 0.2

01 · 정의

test_size는 평가용으로 떼어 둘 비율

02 · 예

0.2면 전체의 20%를 평가용으로

03 · 나머지

나머지 80%가 학습용

비율 선택의 균형

학습 자료량과 시험 정밀도 사이 균형 잡기

01 · 평가용 많이

예: test_size 0.4

시험은 정밀하나 학습 데이터 줄어듦

학습 부족 위험

02 · 평가용 적게

예: test_size 0.1

학습은 넉넉하나 시험이 부실

평가 신뢰도 ↓

03 · 우리 선택

test_size = 0.2

학습 80 / 평가 20

균형 잡힌 표준

random_state — 같은 결과 재현하기

우리 수업의 약속 — `random_state = 42`

01 · 기본 동작

나눌 때 무작위로 섞어 학습용/평가용을 고름

02 · 옵션 효과

`random_state`를 정하면 매번 같은 방식으로 나뉨

03 · 우리 약속

우리 수업은 42로 고정

random_state가 같으면

숫자 42 자체에 특별한 뜻은 없음 — 약속일 뿐

01 · 재현성

같은 random_state면 누가 언제 실행해도 같은 분할

02 · 효과

수업과 복습에서 결과가 어긋나지 않음

03 · 숫자의 의미

숫자 42 자체에 특별한 뜻은 없음 — 약속일 뿐

stratify — 불균형일 때 비율 지켜 나누기

소수 클래스가 평가용에서 사라지는 사고를 막는 장치

01 · 문제

그냥 나누면 학습용과 평가용의 클래스 비율이 틀어질 수 있음

02 · 해결

stratify=y를 주면 두 쪽 모두 원래 비율 유지

03 · 적용

불균형 데이터에서 특히 중요

stratify를 쓰는 우리 설정

이번 모듈의 데이터 분리 설정 — 세 옵션을 모두 적용

01 · 옵션 추가

`train_test_split(..., stratify=y)`

02 · 효과

학습용과 평가용 모두 같은 클래스 비율 유지

03 · 안전장치

소수 클래스(1)가 한쪽에 쏠리는 일을 방지

실습 — failure_soon 라벨 만들기

RUL ≤ 30 규칙으로 failure_soon 컬럼을 직접 생성

01 · 실습 목표

RUL ≤ 30 규칙으로 failure_soon 컬럼을 직접 생성

02 · 사용 데이터

cmapss_fd001_sample.csv

참고

정답은 실습코드 파일을 참고

실습 — 데이터 불러오기

이 시점에는 failure_soon 컬럼이 아직 없는 상태

CODE · LOAD.PY

```
import pandas as pd

df = pd.read_csv('cmapss_fd001_sample.csv')
```

CHECKPOINT

failure_soon 컬럼이 **아직 없음**을 먼저 확인

실습 — 라벨 생성

조건식 → `astype(int)` → 새 컬럼에 담기

CODE · CREATE_LABEL.PY

```
# RUL ≤ 30 조건의 참/거짓을 0/1 정수로 변환해 새 컬럼에 담기  
df['failure_soon'] = (df['RUL'] <= 30).astype(int)
```

한 줄의 의미

각 행마다 조건 판정 → 0/1 변환 → 새 컬럼 추가

실습 — 결과 확인

RUL과 failure_soon을 나란히 비교해 잘 붙었는지 확인

CODE · HEAD.PY

```
# RUL과 failure_soon 두 컬럼을 나란히 출력 (앞 10행)  
df[['RUL', 'failure_soon']].head(10)
```

CHECKPOINT

RUL 30 이하인 행은 1, 큰 행은 0으로 잘 붙었는지 확인

실습 — 경계값 점검

RUL이 정확히 30인 행에서 failure_soon이 1인지 확인

CODE · BOUNDARY.PY

```
# RUL이 정확히 30인 행만 골라 RUL과 failure_soon 출력  
df[df['RUL'] == 30][['RUL', 'failure_soon']].head()
```

CHECKPOINT

failure_soon이 1인지 확인 — 30은 "이하"에 포함

실습 — 클래스 분포 확인

개수와 비율을 직접 보고 불균형을 눈으로 확인

01 · 도구

`value_counts`로 0과 1의 개수와 비율을 확인

02 · 목적

우리 데이터가 불균형임을 직접 눈으로 확인

03 · 다음 단계

먼저 개수, 그다음 비율로 확인

실습 — 개수 세기

0과 1이 각각 몇 개인지 출력

CODE · COUNT.PY

```
# 각 클래스(0, 1)가 몇 개인지 카운트  
df['failure_soon'].value_counts()
```

CHECKPOINT

0이 1보다 훨씬 많은지 확인

실습 — 비율로 보기

`normalize=True` 옵션으로 개수 대신 비율 출력

CODE · `RATIO.PY`

```
# normalize=True를 주면 개수 대신 비율로 보여 줌  
df['failure_soon'].value_counts(normalize=True)
```

CHECKPOINT

1의 비율이 꽤 작음 = **클래스 불균형**

실습 — 불균형 정리

뒤에서 데이터를 나눌 때 `stratify`로 이 비율을 유지

01 · 결과

1(곧 고장)의 비율이 작음 = **클래스 불균형**

02 · 대응

데이터 나눌 때 `stratify`로 비율 유지

실습 — 임계값 바꿔 분포 비교

임계값 선택이 라벨에 주는 영향을 체감하는 실험

01 · 실험 설계

임계값 20, 30, 50으로 1의 개수가 어떻게 변하는지 비교

02 · 목적

임계값 선택이 라벨에 주는 영향을 체감

실습 — 임계값 20

RUL 20 이하인 행의 개수 — 임계값 20일 때 1의 개수

CODE · TH_20.PY

```
# 임계값 20일 때 1의 개수 — 즉 RUL ≤ 20인 행의 수  
(df['RUL'] <= 20).sum()
```

의미

sum()으로 True의 개수를 셈

실습 — 임계값 30과 50

임계값 30과 50일 때 1의 개수를 각각 확인

CODE · TH_30_50.PY

```
# 임계값 30, 50일 때 1의 개수
(df['RUL'] <= 30).sum()
(df['RUL'] <= 50).sum()
```

CHECKPOINT

임계값이 커질수록 1의 개수가 늘어남

실습 — 임계값 영향 정리

같은 데이터인데 임계값만 바뀌어도 라벨이 크게 달라짐

01 · 임계값 ↑

1이 늘어 더 일찍 경고

02 · 임계값 ↓

1이 줄어 임박한 것만 경고

실습 — 누수 체험

모델 없이도 간단한 비교로 누수의 위력을 확인

01 · 가설

RUL만으로 `failure_soon`을 완벽히 되살릴 수 있음을 확인

02 · 결론

RUL이 정답을 그대로 알려 주는 누수임을 체감

실습 — RUL로 라벨 되살리기

모델 없이 RUL만 가지고 failure_soon을 다시 계산

CODE · RECALC.PY

```
# RUL만으로 failure_soon을 다시 계산해 recalc에 담기  
recalc = (df['RUL'] <= 30).astype(int)
```

의미

모델 없이도 RUL만 있으면 정답을 그대로 만들 수 있음

실습 — 원래 라벨과 비교

되살린 라벨과 원래 라벨이 **전부 같은지** 확인

CODE · COMPARE.PY

```
# 모든 행에서 recalc == failure_soon인지 검증
(recalc == df['failure_soon']).all()
```

CHECKPOINT

결과가 **True** — RUL만으로 정답을 100% 맞힘

실습 — 누수의 의미

RUL 하나로 정답이 완벽히 결정 — 입력에 넣으면 커닝

01 · 결론

RUL은 정답을 그대로 알려 주므로 x 에서 제외

02 · 경고

누수가 있으면 학습은 완벽해도 실전은 무력

실습 — x와 y 구성하기

센서만 x에 넣고 RUL은 빼는 게 핵심

01 · 입력

센서 6개를 x로 구성

02 · 정답 + 제외

failure_soon을 y로 · unit_id, cycle, RUL은 제외

실습 — feature 목록 정하기

x에 넣을 센서 6개의 이름을 목록으로 정리

CODE · FEATURES.PY

```
# x에 넣을 센서 6개의 이름을 리스트로 준비
features = ['sensor_2', 'sensor_3', 'sensor_4',
            'sensor_7', 'sensor_11', 'sensor_15']
```

의미

여기서 정한 6개만 x에 들어감

실습 — x 만들기

features 목록으로 센서 컬럼들만 골라 x에 담기

CODE · MAKE_X.PY

```
# features 목록으로 센서 컬럼만 골라 x 생성
X = df[features]
X.head()
```

CHECKPOINT

x에 센서 컬럼만 있고 **RUL**이나 **unit_id**는 **없는지** 확인

실습 — y 만들기

정답인 failure_soon만 골라 y에 담기

CODE · MAKE_Y.PY

```
# 정답인 failure_soon만 골라 y에 담기
y = df['failure_soon']
y.head()
```

의미

y는 0과 1로만 이루어진 단일 컬럼

실습 — x/y 확인

입력과 정답은 **같은 수의 행**을 가져야 함

CODE · SHAPE.PY

```
# x와 y의 크기 확인
X.shape
y.shape
```

CHECKPOINT

두 행 개수가 같은지 확인 — 입력과 정답은 같은 수의 행을 가짐

실습 — 데이터 나누기

배운 세 가지 설정을 모두 적용해 실제로 분리

01 · 목표

train_test_split으로 학습용과 평가용 분리

02 · 설정

test_size 0.2 · random_state 42 · stratify=y

실습 — split 도구 불러오기

scikit-learn에서 데이터를 나눠 주는 도구 불러오기

CODE · IMPORT.PY

```
# scikit-learn의 train_test_split 함수 불러오기
from sklearn.model_selection import train_test_split
```

의미

분리 도구를 호출 가능한 상태로 준비

실습 — 분리 실행

test_size · random_state · stratify 모두 지정해 네 덩어리로 분리

CODE · TRAIN_TEST_SPLIT.PY

```
# X와 y를 학습 80 / 평가 20 비율로 분리
# random_state=42로 결과 재현성 확보, stratify=y로 비율 유지
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

의미

배운 세 옵션을 모두 적용한 분할 한 줄

실습 — 학습용 크기 확인

학습용 입력 `x_train`과 정답 `y_train`의 행 수 확인

CODE · TRAIN_SHAPE.PY

```
# 학습용 입력과 정답의 크기 확인
x_train.shape
y_train.shape
```

CHECKPOINT

전체의 약 **80%**인지 확인

실습 — 평가용 크기 확인

평가용 입력 `x_test`와 정답 `y_test`의 행 수 확인

CODE · TEST_SHAPE.PY

```
# 평가용 입력과 정답의 크기 확인
x_test.shape
y_test.shape
```

CHECKPOINT

전체의 약 **20%**인지 확인

실습 — 분리 정리

네 덩어리가 모두 만들어졌는지 확인

01 · 분할 비율

학습용과 평가용이 약 80 대 20으로 나뉨

02 · 완성

`X_train, X_test, y_train, y_test` 네 덩어리 완성

실습 — 학습/평가 클래스 비율 비교

두 쪽의 클래스 비율을 비교해 stratify의 효과 검증

01 · 확인 목표

학습용과 평가용의 1 비율이 비슷한지 확인

02 · 의의

stratify가 비율을 지켰음을 직접 검증

실습 — 학습용 비율

학습용 정답의 0과 1 비율 확인

CODE · TRAIN_RATIO.PY

```
# 학습용 정답의 0과 1 비율 확인
y_train.value_counts(normalize=True)
```

의미

이 비율을 평가용과 비교

실습 — 평가용 비율

평가용 정답의 0과 1 비율 확인

CODE · TEST_RATIO.PY

```
# 평가용 정답의 0과 1 비율 확인  
y_test.value_counts(normalize=True)
```

CHECKPOINT

학습용과 평가용의 1 비율이 거의 같으면 **stratify 성공**

실습 — stratify 효과 정리

학습용과 평가용 모두 원래의 불균형 비율을 그대로 유지

01 · 결과

두 쪽 모두 원래 비율을 유지 = stratify 성공

02 · 의미

불균형 데이터를 공정하게 나누는 핵심 장치

실습 — test_size 바꿔 관찰

비율 설정이 분할 크기에 주는 영향을 직접 관찰

01 · 실험

test_size를 바꾸면 학습용/평가용 크기가 달라짐

02 · 목적

비율 설정이 분할에 주는 영향을 확인

실습 — test_size 0.3으로

test_size를 0.3으로 바꿔 다시 분할

CODE · SPLIT_03.PY

```
# test_size를 0.3으로 바꿔 다시 분할 → 평가용이 전체의 30%
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
X_te.shape
```

의미

평가용이 전체의 30%가 됨

실습 — 크기 비교

test_size 0.2일 때와 0.3일 때의 학습용 크기를 비교

CODE · COMPARE_SHAPE.PY

```
# 0.2일 때 학습용과 0.3일 때 학습용 크기 비교
X_train.shape
X_tr.shape
```

CHECKPOINT

test_size를 키우니 학습용이 줄어듦

실습 — 모듈 정리

모델이 먹을 수 있는 형태로 데이터가 완전히 준비됨

STEP 1

라벨 생성

$RUL \leq 30$ 으로
failure_soon 생성



STEP 2

클래스 확인

개수·비율로 불균형 확
인



STEP 3

x/y 구성

센서 6개와 정답으로
분리



STEP 4

학습/평가 분리

train_test_split + stratify



STEP 5

다음 모듈

랜덤 포레스트 학습과
예측

모듈 완료

다음 모듈에서 이 데이터로 **랜덤 포레스트** 학습